

# EMBEDDED SYSTEM AND DESIGN

## EXPERIMENT NO. 5

# REAL-TIME CLOCK

Name: *Raphael S. Nano*

Course/Section: *CPE161P / AI*

Date of Performance: *December 12-14, 2023*

Date of Submission: *December 14, 2023*

*Ma'am Cyrel Manlises*  
Instructor

# DISCUSSION

*A clock is just an instrument for measuring time. At the moment, consumers can choose between analog and digital. A real-time clock, according to one common definition, moves at regular intervals indicated by seconds, minutes, and hours, and it is synchronized with the world's standard time. Clocks are an essential part of our everyday lives, acting as constant companions on people's time travels. With that being said, a digital real-time clock was created using Arduino Uno as the controller, seven segment display and LCD for the output of the clock, and 5 buttons for the adjustments of the clock.*

*The objectives of experiment 5 are as follows:*

*1. To demonstrate how a real-time clock works (hours-minutes-seconds).*

2. To be able apply what you have learned in logic circuits and switching theory, microprocessors, and CPE160P.

3. To be able to apply real-time embedded system using arduino kit.

The materials utilized for this experiment include the Arduino Uno, the 4-digit seven segment display, the Liquid Crystal Display with an integrated IC, and 5 push buttons.

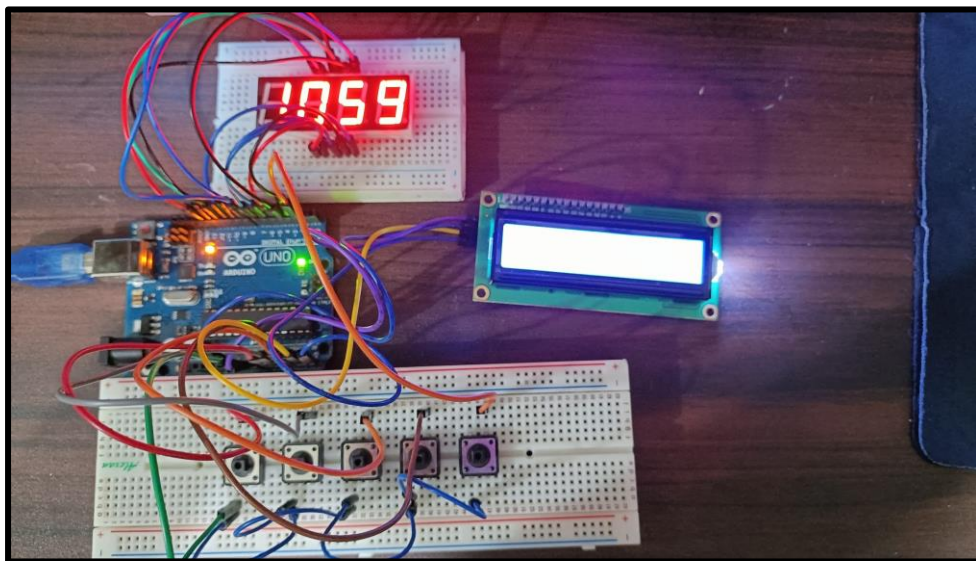


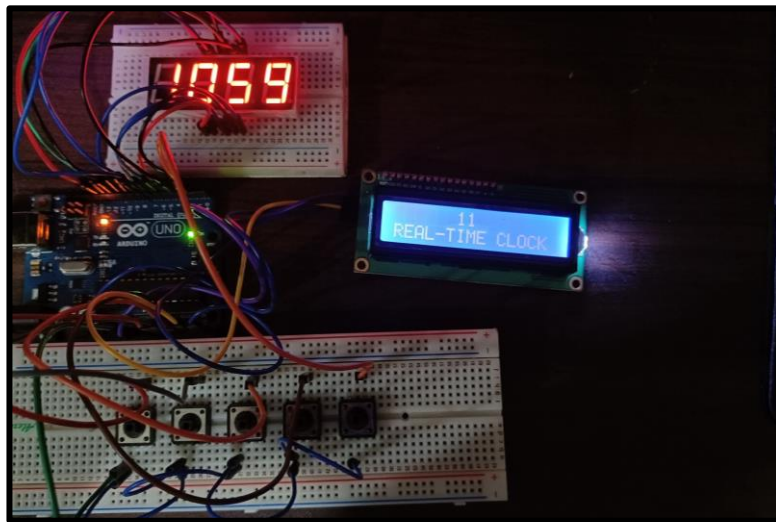
Figure 1. Major Components for the Experiment

For the external libraries, `LiquidCrystal_I2C.h` and `SevSeg.h` was utilized for the LCD and the 4-digit seven segment display, respectively.

```
#include <SevSeg.h>
#include <LiquidCrystal_I2C.h>
```

*Figure 2. External Libraries Included*

*Experiment 5 mainly aims to replicate a digital clock that works in real-time, wherein the current hour, minute, and second is being displayed for the output. With the use of the 5 push buttons, the user may manually adjust the clock to the current time.*



*Figure 3. Final Output of the Experiment*

*The activity may be broken into three sections: constant ticking of the clock, increasing minutes or hours, and decreasing minutes or hours. The output should have a tick rate of exactly 1000 milliseconds each tick, or one second, wherein the seconds are displayed on the LCD. At the same*

time, the minute ticks on the 7-segment display should be increasing, such that starting at 1:00, the numbers should be 1:01, 1:02, 1:03, and so on. As a result, for the seven-segment display, the potential clock combinations were first saved in an array designated `numarrgen()`, standing for number array generator, and the stored numbers would be shown in the 7-segment after a total of 60 seconds have passed.

```
void numarrgen() {  
    int hrs;  
    int minutes;  
    for (int i = 1; i < 13; i++) {  
        hrs = i;  
        for (int k = 0; k < 60; k++) {  
            minutes = k;  
            numarray[timenum] = ((hrs * 100) + minutes);  
            timenum++;  
        }  
    }  
}
```

Figure 4. `numarrgen()` function code

Furthermore, the delay was achieved by utilizing the `millis()` method, which returns the number of milliseconds the program has been executing. This function assures the program-based timing of a minute delay without compromising the output in the 7-segment.

```
void msdelay(int delNum) {  
    currentms = millis();  
    startms = millis();  
    while (currentms - startms < delNum) {  
        currentms = millis();  
        sevseg.refreshDisplay();  
    }  
}
```

Figure 5. msdelay() function code

*Setting up an accurate clock requires the ability to raise and decrease the minutes and hours in relation to real-time. The seconds of the clock continues to tick and is not allowed to be adjusted. The notion of increased time adjustment begins with the fifth button. The first four buttons are activated while the fifth button is kept down. The first button, which handles the hourly increment, was written in such a way that it will rise from 1 to 12 but will return to 1 when increased again. The second button, the minute incrementing handler, increments the value of minutes from 0 to 59 and returns to 0.*

```

void loop() {
  lcd.clear();
  numarrgen();
  lcd.setCursor(0, 1);
  lcd.print("REAL-TIME CLOCK");
  int i = 0;
  int sec = 0;
  while (i < 721) {
    sec++;
    if (sec == 60)
    {
      sec = 0;
      i++;
    }
    dispnum = numarray[i];
    sevseg.setNumber(dispnum);
    sevseg.refreshDisplay();
    wantedNum = dispnum;
    while (digitalRead(2) == LOW) {
      wantedNum = adjust(dispnum);
      for (int j = 0; j < 721; j++) {
        if (wantedNum == numarray[j]) {
          i = j;
          break;
        }
      }
    }
    if (sec < 10) {
      lcd.setCursor(6, 0);
      lcd.print("0");
      lcd.print(sec);
    } else {
      lcd.setCursor(6, 0);
      lcd.print(sec);
    }
    msdelay(1000);
  }
}

```

Figure 6. Main Loop code

*The third button, which handles hour decrement, subtracts the number of hours from 12 to 1 and then back to 12. Finally, the fourth button decrements minutes in a sequence that goes from 59 to 0 and back to 59. The logic in the code takes into account numerous if-else statements when*

*determining which button is clicked and the current value of the number shown. It is possible to change the time by pressing the fifth button.*



(a)



(b)



(c)

Figure 7. Sample Final Outputs of Experiment 5

# CONCLUSION

*In conclusion, the digital clock constructed works in real-time and its operation is shown in three functions. The first function is the constant ticking of time, wherein the clock is continuous and is synchronized with the seconds, minutes, and hours of a standard clock. The second and third functions are the manual incrementing and decrementing of time, in the case that the clock had started off differently as compared to real-time. Proven by the success in creating the real-time clock with the use of various functions, real-time modules, and circuit connections, topics in logic circuits and switching theory, microprocessors, and especially CPE160P were fully implemented. Hence, a digital clock was*

*created and is operating appropriately as an application of a real-time embedded system.*